1. Command Table (Top 20)

Command	Flags Used / Syntax	Description
ls	ls -l	List dir contents. -l shows permissions, onr, size.
pwd	pwd	Print Working Directory. Shows your current location.
mkdir	Mkdir <name>	Make Directory. Creates a new folder.
touch	touch <file>	Create an empty file or update its timestamp.
cp	cp -r <src> <dest>	Copy files. -r is used to copy directories recursively.
mv	mv <old> <new>	Move or Rename files/directories.
rm	rm -r <dir>	Remove files. -r deletes a directory and its contents.
cat	cat <file>	Concatenate and display the entire file content.
grep	grep "text" <file>	Search for a specific text pattern within a file.
chmod	chmod 600 <file>	Change mode/permissions (e.g., 600 for read/write user only).
ps	ps	Process Status. Snapshot of current running processes.
kill	kill <PID>	Terminate a process using its Process ID (PID).
find	find . -name "name"	Search for files in a directory hierarchy.
du	du -h	Disk Usage. Checks directory size. -h makes it human-readable.
man	man <command>	Manual. Displays the help manual for a command.
head	head <file>	Output the first part (usually 10 lines) of a file.
history	history	View the list of previously executed commands.
tar	tar -czf, tar -xf	Archive utility. -czf to Create Gzip File, -xf to eXtract File.
wc	wc <file>	Word Count. Counts lines, words, and characters.

2. Bash Syntax Guide

Derived from your scripts combined_all.sh (Task 1A/2B) and check_permissions.sh.

If/Else Logic

Bash uses [ ] for test conditions. **Spaces inside the brackets are mandatory.**

- **Comparing Numbers:**
  - **Syntax:** if [$var -op value]; then ... fi
  - **Operators:** -eq (equal), -ne (not equal), -gt (greater than), -lt (less than), -ge (greater/equal), -le (less/equal).
  - Example (from combined_all.sh):

```
Bash
if [ $n -le 1 ]; then
    echo "$n is NOT a prime number"
elif [ $r -eq 0 ]; then
    echo "$n is Even"
fi
```

- **Comparing Strings & Files:**
  - **Syntax:** if ["$str" = "val"] or file tests like -e (exists), -z (empty string).
  - Example (from check_permissions.sh):

```
Bash
if [ -z "$1" ]; then            # Check if argument is empty
    echo "Usage: $0 <filename>"
fi
if [ ! -e "$filename" ]; then     # Check if file does NOT exist
    echo "Error: File not found."
fi
```

Loops

Your scripts primarily use while loops for iteration.

- **While Loop:**

- Example (Fibonacci from combined_all.sh):

```
Bash
count=2
while [ $count -lt $n ]; do    # Loop while count < n
    c=$((a + b))
    echo -n " $c"
    a=$b
    b=$c
    count=$((count + 1))       # Increment counter
done
```

User Input

- **Syntax:** read <variable_name>
- Example:

```
Bash
echo -n "Enter a number: "  # -n prevents a new line
read n                      # Stores input in variable $n
```

3. One-Liners & "Clever" Logic

Patterns extracted from your check_permissions.sh, combined_all.sh, and Lab Manual instructions.

- **Quick Arithmetic:**

Use $((...)) for math without external commands.

```
Bash
r=$((n % 2))      # Modulo operation to find remainder
half=$((n / 2))   # Integer division
```

- **File Extension/Type Check (file_type.sh logic):**

While the full script wasn't in the snippet, the lab manual describes using **Case Statements** or file tests to check extensions. The standard "clever" way to handle multiple extensions is:

```
Bash
case "$filename" in
    *.jpg|*.png) echo "Image File" ;;
    *.txt)       echo "Text File" ;;
    *.c)         echo "C Source File" ;;
    *)           echo "Unknown Type" ;;
esac
```

Alternatively, extract the extension directly: ext="${filename##*.}"

- **Permission Checks (The "Silent" Check):**

From check_permissions.sh, checking flags directly:

```
Bash
[ -r "$filename" ] && echo "Readable: Yes"  # One-line logic using &&
[ -w "$filename" ] && echo "Writable: Yes"
[ -x "$filename" ] && echo "Executable: Yes"
```

1. Key System Calls & Functions

fork()

Creates a new process (Child) by duplicating the calling process (Parent).

- **Returns:** 0 to the Child, PID of the Child to the Parent, -1 on failure.
- **Snippet:**

```
C
pid_t pid = fork();
if (pid < 0) {
    perror("Fork failed");
    exit(1);
}
else if (pid == 0) {
    // Child Process
    printf("I am the Child (PID: %d)\n", getpid());
    // Do child tasks here...
}
else {
    // Parent Process
    printf("I am the Parent (PID: %d), my Child is %d\n", getpid(), pid);
    // Parent continues...
}
```

exec() Family

Replaces the current process image with a new program. The process ID remains the same, but the memory is overwritten.

- **Common Variants:** execl, execv, execvp.
- **Snippet (using execl):**

```
C
// Syntax: execl(path, arg0, arg1, ..., NULL);
// arg0 is usually the name of the executable itself.
printf("Replacing this process with 'ls -l'...\n");
execl("/bin/ls", "ls", "-l", NULL);
// This line is ONLY reached if execl fails
perror("Exec failed");
exit(1);
```

wait() / exit()

Used to sync processes & manage termination status.

- **exit(status):** Terminates the process and returns status to the OS (0 = success).
- **wait(&status):** Suspends the parent until one child terminates. Preventing "Zombie" processes (dead processes waiting for their parent to read their exit status).
- **Snippet:**

```
C
```

```
pid_t pid = fork();
if (pid == 0) {
    printf("Child working...\n");
    sleep(2);     // Simulate work
    exit(0);      // Terminate with success
} else {
    int status;
    wait(&status); // Block until child exits
    if (WIFEXITED(status))
        printf("Child exited with status: %d\n", WEXITSTATUS(status));
}
```

getpid() / getppid()

Retrieves process identifiers.

- **getpid():** Returns PID of the calling process.
- **getppid():** Returns the Process ID of the parent of the calling process.
- **Snippet:**

```c
printf("My PID: %d\n", getpid());
printf("My Parent's PID: %d\n", getppid());
```

2. Code Pattern: The "Process Management" Template

Use this template to start any C assignment involving processes. It includes all necessary headers found in your forkFunc.c, processID.c, and priority_logging.c files.

```c
/* * Template: Process Management & System Calls
 * Includes headers for fork(), wait(), pid types & standard I/O.
 */
#include <stdio.h>     // For printf, perror
#include <stdlib.h>    // For exit, rand
#include <unistd.h>    // For fork, exec, getpid, sleep, usleep
#include <sys/types.h>  // For pid_t definition
#include <sys/wait.h>   // For wait, waitpid macros (WIFEXITED, etc.)
#include <errno.h>     // For error handling (optional but good practice)
int main() {
    // 1. Variable Declarations
    pid_t pid;
    int status;
    // 2. Forking a new process
    pid = fork();
    // 3. Error Handling
    if (pid < 0) {
        perror("Fork failed");
        exit(EXIT_FAILURE);
    }
    // 4. Child Logic
    if (pid == 0) {
        printf("Child Process (PID: %d) started.\n", getpid());
        // TODO: Add Child code here (e.g., exec() or computation)
        exit(EXIT_SUCCESS); // Always exit explicitly
    }
    // 5. Parent Logic
    else {
        printf("Parent Process (PID: %d) waiting for Child (%d)...\n", getpid(), pid);
        // Wait for child to prevent Zombies
        wait(&status);
        if (WIFEXITED(status))
printf("Child finished normally with status %d.\n", WEXITSTATUS(status));
        printf("Parent resuming execution.\n");
    }
    return 0;
}
```

Based on your synchronization files (diningPhilosophers.c, readerWriterSynchronization.c, h2obond.c, and lab5_mutex_semaphores.c), here is **Part 3** of your Cheat Sheet.

1. Thread Syntax

- **Create a Thread:**

```c
#include <pthread.h>
pthread_t thread_id;
// Syntax: pthread_create(&id, attr, function, arg);
pthread_create(&thread_id, NULL, thread_function, (void*)&arg);
```

- **Join a Thread (Wait for it to finish):**

```c
// Syntax: pthread_join(id, retval);
pthread_join(thread_id, NULL);
```

2. Locks & Semaphores

Mutex (Locks)

Used for **mutual exclusion** (only one thread accesses a critical section at a time).

- **Header:** #include <pthread.h>
- **Syntax:**

```c
pthread_mutex_t lock;
// 1. Initialize
pthread_mutex_init(&lock, NULL);
// OR: pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER; (Global static only)
```

```
// 2. Lock (Start Critical Section)
pthread_mutex_lock(&lock);
// ... Critical Section Code ...
// 3. Unlock (End Critical Section)
pthread_mutex_unlock(&lock);
// 4. Destroy (Cleanup)
pthread_mutex_destroy(&lock);
```

Semaphores

Used for **signaling** and **resource counting** (allowing limited concurrent access).

- **Header:** #include <semaphore.h>
- **Syntax:**

```
C
sem_t my_sem;
// 1. Initialize
// Syntax: sem_init(&sem, pshared, value);
// pshared=0 (threads only), value=Initial Count
sem_init(&my_sem, 0, 1);
// 2. Wait (Decrement / Block if 0)
sem_wait(&my_sem);
// ... Critical Section Code ...
// 3. Post (Increment / Signal)
sem_post(&my_sem);
// 4. Destroy
sem_destroy(&my_sem);
```

3. Problem Logic Summaries

- **Dining Philosophers (dining_philosophers.c):** The deadlock is prevented by **Resource Hierarchy** (ordering) and **Limiting Concurrency**.
    - **Logic:** Philosophers pick up forks in a strictly defined order (e.g., lower ID fork first, then higher ID) to avoid a circular wait. Additionally, a semaphore (room_sem) limits the number of philosophers at the table to N-1, ensuring at least one person can always eat.
- **Reader-Writer (readerWriterSynchronization.c):** This problem prioritizes data consistency by allowing multiple readers but only one exclusive writer.
    - **Logic:** A mutex (wrt) ensures exclusive access for writers. A separate mutex protects the read_count variable; the first reader to arrive locks wrt to block writers, and the last reader to leave unlocks wrt to allow writers to proceed.
- **H2O Bond (h20bond.c):** This ensures that water molecules are formed only when exactly 2 Hydrogen and 1 Oxygen threads are ready.
    - **Logic:** The code uses two semaphores (h_sem initialized to 2, o_sem initialized to 1) or a barrier mechanism. Hydrogen threads wait for an Oxygen signal and vice versa, and a barrier (or counter) ensures the bond() function is called only after the correct group (H-H-O) has formed.

1. Banker's Algorithm

Source: Bankers_Algorithm.c

Safety Algorithm Steps

1. **Work = Available**: Initialize a temporary array Work with the currently available resources.
2. **Finish[i] = false**: Mark all processes as unfinished initially.
3. **Find a Process i**: Look for a process that has not finished (Finish[i] == false) and whose needs can be met by the current available resources (Need[i] <= Work).
4. **Execute & Release**: If such a process exists:
    - Simulate execution (it finishes).
    - Release its allocated resources back to Work: Work = Work + Allocation[i].
    - Mark Finish[i] = true.
    - Repeat Step 3.
5. **Check Safety**: If all processes are marked Finish = true, the system is in a **Safe State**.

Matrices & Formulas

- **Allocation[i][j]**: Number of res of type j currently held by process i.
- **Max[i][j]**: Max demand of process i for res j.
- **Available[j]**: Num of res of type j currently free in the system.
- **Need[i][j]**: Remaining resources process i needs to complete its task.
    - **Formula:** Need[i][j] = Max[i][j] - Allocation[i][j].

2. CPU Scheduling

Source: cpu-scheduling-simulation/main.cpp and Scheduler.hpp

Preemptive SJF vs. Priority Scheduling

- **Preemptive SJF (Shortest Job First):**
  - **Criteria:** Selects the process with the **shortest remaining burst time**.
  - **Preemption:** If a new process arrives with a shorter burst time than the current running process, the CPU is taken away (preempted) and given to the new one.
- **Priority Scheduling:**
  - **Criteria:** Selects the process with the **highest priority** (typically lower number = higher priority).
  - **Mechanism:** Your implementation in main.cpp reads a priority integer for each process. The scheduler picks the available process with the best priority value.

Performance Formulas

- **Turnaround Time (TAT):** Total time from submission to completion.
  - TAT = Completion Time - Arrival Time
- **Waiting Time (WT):** Total time spent waiting in the ready queue.
  - WT = Turnaround Time - Burst Time

3. Deadlock Detection

Source: deadlock_detection.cpp

In a system with **Single-Instance Resources** (where each resource type has only one instance), a deadlock exists if and only if the **Resource Allocation Graph (RAG)** contains a **Cycle**.

- **Condition Checked:** The code builds a directed graph where nodes are Processes and Resources. It runs a **Depth First Search (DFS)** looking for a "back edge" (an edge pointing to a node currently in the recursion stack).
- **Logic:** If dfs(u) encounters a neighbor v that is already visited in the current recursion path (color[v] == 1), a cycle is detected, confirming a **DEADLOCK**.

1. Quick Compilation Guide

File Type	Command	Explanation
Standard C (Bankers_Algorithm.c, forkFunc.c)	gcc forkFunc.c -o fork	Standard compilation.
Threads & Semaphores (diningPhilosopher.c, h20bond.c)	gcc -pthread diningPhilosopher.c -o dining	**Crucial:** -pthread links the POSIX thread library. Without it, you will get "undefined reference to pthread_create" errors.
Simple C++ (Any single .cpp file)	g++ file.cpp -o file	Use g++ for C++ files.
Complex C++ Project (cpu-scheduling-simulation)	g++ -o vibe-cpu main.cpp src/*.cpp -I include	**Multi-file compilation:** 1. src/*.cpp: Compiles all source files. 2. -I include: Tells the compiler to look for headers (.hpp) in the include folder.

2. Common Pitfalls & Exam Checklist

Derived from analyzing transactionServer.c and raceCondition.c.

A. The "Pass-by-Reference" Loop Bug

- **The Error:** passing the address of the loop variable i directly to a thread.

```c
// BAD: By the time thread starts, 'i' has changed!
pthread_create(&t, NULL, func, &i);
```

- **The Fix (seen in transactionServer.c):** Use a dedicated array to store IDs so the memo address remains valid and unique for each thread.

```c
int ids[NUM_CLIENTS]; // Dedicated array
for (int i=0; i<N; i++) {
    ids[i] = i;
```

```
    pthread_create(&client[i], NULL, func, &ids[i]); // Pass stable address
}
```

B. The "Infinite Block" (Forgetting to Unlock)

- **The Error:** Locking a mutex or waiting on a semaphore but failing to unlock/post it before a return or break statement.
- **Eg Cntxt:** In transactionServer.c, if client_thread returns early without calling sem_post(&serverSlots), the semaphore cnt never increases. After 3 clients (the cap), the server will **deadlock** & ignore all future clients.

C. The "Implicit Declaration" (Missing Headers)

- **The Error:** Using functions like printf, sleep, or sem_wait without imports.
- **The Fix:** Check your top-level imports.
    - printf $\rightarrow$ #include <stdio.h>
    - fork, sleep, write $\rightarrow$ #include <unistd.h>
    - bool, true/false $\rightarrow$ #include <stdbool.h> (Seen in Bankers_Algorithm.c).
    - sem_t $\rightarrow$ #include <semaphore.h> (Seen in readerWriterSynchronization.c).

D. Race Conditions (The "Missing Lock")

- **The Error:** Modifying a global var (like cntr) from multiple threads without a mutex.
- **Symptom:** The final value is less than expected (e.g., 100,000 + 100,000 = 145,230 instead of 200,000).
- **The Fix:** Wrap the increment in a lock/unlock block, as demonstrated in raceConditionUsingMutex.c.

1. FCFS (First Come First Serve)

Core Logic: Process in order of arrival. Update time, then calculate TAT & WT.

```
C++
// struct Process {int id, at, bt, ct, tat, wt;};
int currentTime = 0;
for (int i = 0; i < n; i++) {
    // 1. Handle idle CPU if process hasn't arrived
    if (currentTime < p[i].at)
        currentTime = p[i].at;
    // 2. Execute Process
    p[i].ct = currentTime + p[i].bt;   // Completion = Start + Burst
    // 3. Calculate Metrics
    p[i].tat = p[i].ct - p[i].at;      // TAT = CT - AT
    p[i].wt  = p[i].tat - p[i].bt;     // WT = TAT - BT
    // 4. Update Time
    currentTime = p[i].ct;
}
```

2. SJF (Non-Preemptive)

Core Logic: Sort by Burst Time (BT) and execute. (Note: Simple sorting assumes all arrived at t=0. For varying AT, filter ready processes first).

```
C++
// Using <algorithm> header
// 1. Sort all processes by Burst Time (Asce)
std::sort(p.begin(), p.end(), [](Process a, Process b) {
    return a.bt < b.bt;
});
// 2. Calculate Completion (Standard Loop)
int currentTime = 0;
for (int i = 0; i < n; i++) {
    currentTime += p[i].bt;
    p[i].ct = currentTime;
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
}
```

3. Round Robin

Core Logic: Use a queue & a time quantum (TQ). If rem > TQ, execute for TQ and re-queue. Else, finish.

```
C++
// queue<int> q; q.push(initial_indices);
// int t = 0;
while (!q.empty()) {
    int i = q.front();
    q.pop();
    // Case A: Process needs more time than Quantum
    if (p[i].rem > TQ) {
        t += TQ;
        p[i].rem -= TQ;
        // Check for new arrivals here before re-queuing!
        q.push(i); // Re-queue current process
    }
    // Case B: Process finishes
    else {
        t += p[i].rem;
```

```
        p[i].rem = 0;
        p[i].ct = t;
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;
    }
}
```

1. Producer-Consumer Problem

Source: lab5_mutex_semaphores.c

This solution uses two semaphores (empty and full) to track buffer space and a mutex (mutex) for exclusive access to the buffer index.

```C
// Shared Variables
sem_t empty; // Initialized to BUFFER_SIZE
sem_t full;  // Initialized to 0
pthread_mutex_t mutex; // Initialized to 1
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
void *producer(void *pno) {
    int item;
    for(int i = 0; i < MaxItems; i++) {
        item = rand(); // Produce an item
        sem_wait(&empty);          // Wait for empty slot
        pthread_mutex_lock(&mutex); // Lock critical sec
        buffer[in] = item;          // Place item
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);// Unlock
        sem_post(&full);            // Signal full slot
    }
}
void *consumer(void *cno) {
    int item;
    for(int i = 0; i < MaxItems; i++) {
        sem_wait(&full);            // Wait for filled slot
        pthread_mutex_lock(&mutex);  // Lock critical section
        item = buffer[out];         // Consume item
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);// Unlock
        sem_post(&empty);           // Signal empty slot
    }
}
```

2. Dining Philosophers Problem

Source: dining_philosophers.c

To avoid deadlock, this implementation typically uses a "resource hierarchy" (pick up lower ID fork first) or a semaphore to limit the number of philosophers eating. The uploaded file uses a semaphore room_sem to limit concurrency to N-1.

```C
sem_t room_sem;   // Initialized to N - 1 (Limit eaters)
sem_t chopstick[N]; // Initialized to 1 each
void *philosopher(void *num) {
    int id = *(int *)num;
    sem_wait(&room_sem);       // 1. Enter room (Deadlock prevention)
    sem_wait(&chopstick[id]);   // 2. Pick up Left Fork
    sem_wait(&chopstick[(id + 1) % N]); // Pick R8 Frk
    // EAT ...
    sem_post(&chopstick[(id + 1) % N]);   //Put R8 Frk
    sem_post(&chopstick[id]);  //  Put down Left Fork
    sem_post(&room_sem);       // 6. Leave room
}
```

3. Reader-Writer Problem

Source: readerWriterSynchronization.c

This sln gives priority to readers (Starve Writers). The **First Reader** locks the database for everyone, and the **Last Reader** unlocks it. Writers just need the db_mutex.

```C
sem_t db_mutex;    // Controls access for Writers
sem_t x;           // Protects read_count
int read_count = 0;
void *reader(void *rno) {
    // Entry Section
    sem_wait(&x);
    read_count++;
    if (read_count == 1) {
        sem_wait(&db_mutex); // First reader locks DB from writers
    }
    sem_post(&x);
    // READING ...
    // Exit Section
    sem_wait(&x);
    read_count--;
    if (read_count == 0)
        sem_post(&db_mutex); // Last reader releases DB for writers
    sem_post(&x);
}
void *writer(void *wno) {
    sem_wait(&db_mutex); // Wait for X-access
    // WRITING ...
    sem_post(&db_mutex);     // Release access
}
```

1. Safety Algorithm Loop//banker's algo

This loop repeatedly checks if any unfinished process can satisfy its resource needs with the currently available work resources.

```C
int Work[m];
```

```
for(int i=0; i<m; i++) Work[i] = Available[i]; // Initialize Work = Available
int Finish[n];
for(int i=0; i<n; i++) Finish[i] = 0; // Init Finish = false
int count = 0;
while (count < n) {
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (Finish[i] == 0) { // If process is not finished
            int j;
            // Check if Need <= Work for all resources
            for (j = 0; j < m; j++)
                if (Need[i][j] > Work[j])  break;
            if (j == m) {
                // Execute Process: Release resources
                for (int k = 0; k < m; k++)
                    Work[k] += Allocation[i][k];
                Finish[i] = 1; // Mark as finished
                found = 1;
                count++;
            }
        }
    }
    if (found == 0)
        break;  // Sys is NOT Safe (Deadlock possibility)
}
```

2. Request Resource Logic

This logic tentatively grants a request and then runs the safety check to see if the resulting state is valid.

```
C
// Assume 'p' is the process index making the req
// 'Request[m]' is the array of requested resources
int j;
// 1. Check if Request <= Need
for (j = 0; j < m; j++)
    if (Request[j] > Need[p][j])
        return -1; //Err: Process exceeded it's max claim
// 2. Check if Request <= Available
for (j = 0; j < m; j++)
    if (Request[j] > Available[j]) {
        // Err: Resources not currently available (Wait)
        return -1;
    }
// 3. Tentative Allocation
for (j = 0; j < m; j++) {
    Available[j] -= Request[j];
    Allocation[p][j] += Request[j];
    Need[p][j] -= Request[j];
}
// 4. Run Safety Algorithm (Code from Part 1)
if (isSafe()) return 1; // Req Granted permanently
else {
    // System Unsafe: Rollback changes
    for (j = 0; j < m; j++) {
        Available[j] += Request[j];
        Allocation[p][j] -= Request[j];
        Need[p][j] += Request[j];
    }
    // Request Denied (Wait)
    return 0;
}
```